Advanced Computer Architecture
Homework 1: Multithreading AlphaBot2

Philipp Dietle
Matriculation ID: 4740093

Lorenz Scherrer
Matriculation ID: 4749927

Gioele Pettarini
Matriculation ID: 4767919

Group: Uebung 2 Group 1 Robot: 8

1 Introduction

The goal of this assignment is to coordinate the AlphaBot2's main tasks so that the robot can complete a round trip on the black track while also reacting to objects in its environment. The required tasks are:

1. continuously following the black line without veering off the track;
2. detecting obstacles with the infrared sensors and buzzing once, twice, or three times for the first, second, or third detected object;
3. recognizing a shoe, a bottle, and a mug with the camera, then setting the RGB LED to red, yellow, or green respectively.

The main challenge is that these tasks have very different timing requirements. Line following must react frequently to keep the robot stable, while camera-based object recognition is much slower and can block the control loop if executed sequentially.

2 Naive approaches

Our first version kept the whole behavior in one loop. The robot read the line sensors, checked the infrared sensors, ran camera recognition, and then updated the outputs. On the track this failed quickly. During a recognition step the motors kept their previous command for too long, so the robot started drifting before the next line correction.

We then tried to fix the drift by changing only the line-following controller. We added integral and derivative terms and tuned the constants, but the result was not stable enough. The extra terms sometimes made the corrections too strong. They also did nothing while recognition was blocking the loop. After that test we used a simpler proportional controller again, because the timing problem was larger than the controller-tuning problem.

After that we split the program into Python threads for driving, camera recognition, and infrared obstacle detection. The code matched the three tasks better, but the robot still had pauses. The vision thread competed for CPU time with the driving thread, and Python's Global Interpreter Lock (GIL) limits how much CPU-bound Python code can run at the same time. We also had small coordination bugs in the first threaded versions: the stop flag and object counter were shared informally, and the buzzer loop could keep running while the rest of the robot should already have moved on.

We also tested separate processes to move the expensive recognition work away from the control loop. That avoided the GIL for recognition, but the robot hardware did not fit this version well. The camera, GPIO pins, model, and AlphaBot object could not be treated as shared objects. Each process had its own memory, so state such as the stop condition, the detected-object count, and the latest recognition result had to be passed explicitly. The first multiprocessing version was harder to debug than the threaded one and overall ended up not working.

For the final version, we kept one rule from these failed attempts: the driving loop should do only line sensing and motor updates. Recognition, buzzing, and LED updates can run outside that loop, but they should exchange only the few values the driver actually needs.

3 Benchmarking

We already knew that object recognition was the heaviest computation, but we wanted real numbers to use when designing the final solution. For this reason, we benchmarked the main operations. The measured runtimes are summarized here.

Task	Iterations	Average time per iteration
Line following	100	0.009808 s
Object recognition	30	0.245612 s
Infrared obstacle check	100	0.000425 s

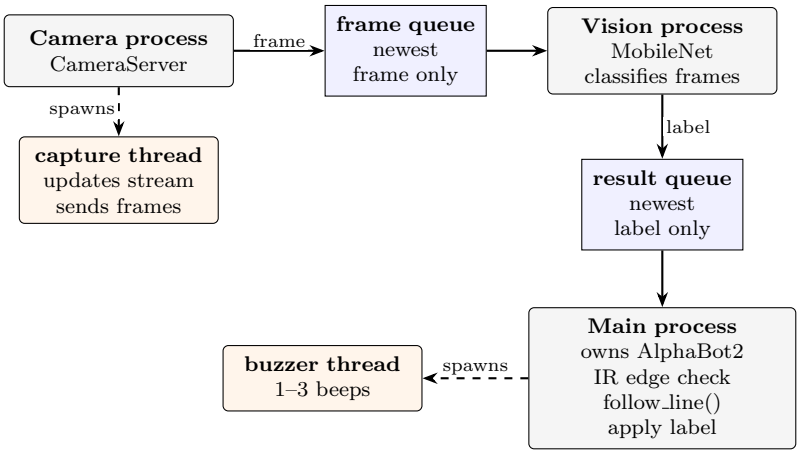
The measurements confirmed our expectation and gave us a concrete starting point for the redesign. Object recognition takes about 0.25 seconds per iteration, while line following and infrared checking are much shorter. In the sequential version, every recognition step therefore skipped many possible line-following updates, which explains the drifting behavior we observed on the track.

4 Solution Design

The final program keeps the fast robot control in the main process and moves the camera work out of it. The main process creates the `AlphaBot2` object only after the camera and vision processes have reported that they are ready. GPIO writes, PWM motor commands, infrared reads, buzzer control, and LED updates stay in one place instead of being spread across processes.

Camera capture and recognition use two separate processes. The camera process starts CameraServer with a frame queue. We changed CameraServerClass.py so it can receive this optional queue. Its capture thread still updates the latest frame for the web stream, and it also pushes a frame into the queue. The queue has size one, so an old frame is dropped if the vision process is still busy. We prefer a recent frame over a backlog of old frames.

The vision process loads MobileNet once. It limits Torch to one thread, reads frames from the frame queue, and maps ImageNet IDs to `shoe`, `bottle`, or `mug`. It writes the newest result to a result queue, also with size one. The main loop checks that queue without blocking. When a label is available, the LED is set according to the assignment: shoe red, bottle yellow, and mug green.



Obstacle detection remains in the main process because it is cheap and tied to the buzzer. We keep `obstacle_was_present` to count only the edge of an obstacle; otherwise the same object would be counted many times while it stayed in front of the sensor. The buzzer pattern runs in a short thread, so beeping does not stop `follow_line` from being called.

We use processes only for the two parts that can delay driving: grabbing frames and running the neural network. The loop that steers the robot stays fast: infrared check, possible buzzer trigger,

`follow_line`, optional LED update, and a 1 ms sleep. The queues carry frames and classification dictionaries; they never carry the `AlphaBot2` object.

5 Results

The final implementation should be evaluated against the assignment requirements:

1. ability to follow the track with at most one slight deviation;
2. correct recognition of the shoe, bottle, and mug;
3. correct buzzer behavior for the first, second, and third detected object;
4. correct RGB LED colors: red for the shoe, yellow for the bottle, and green for the mug;
5. round-trip time on the track.

Compared with the sequential implementation, the expected improvement comes from preventing camera recognition from blocking the line following. The benchmark results support this design decision because recognition is approximately 25 times slower than one line-following iteration. In comparison to the native approach the multithreaded variant works significantly better. Keeping the line following and obstacle detection in the same thread works good because of the sort runtime of the tasks. Another advantage of this approach is that the hardware tasks are done by the same thread which reduces latency of data movement between the cores. Splitting the object recognition into a camera process and a vision process is also advantageous because it splits the heavy workload across multiple processes, which reduces latency through parallelism.

6 Conclusion

The sequential implementation showed that object recognition cannot be placed directly in the main control loop without harming navigation. Benchmarking identified camera inference as the dominant cost, while line following and infrared checks were comparatively lightweight. The final solution therefore separates the time-critical control loop from slower background tasks and uses shared state only for the information needed by the buzzer and LED behavior. Using multiprocessing for the buzzing and LED behavior would be more complicated because Proses copy objects in there initiation arguments which would lead to errors because the hardware can only be controlled by one robot controller instance. Using two processes for the object recognition is good because the two processes have no shared variables besides the image queue.